LocalCloudConfigurator

System Design Description

Abstract
This document describes the implementation of the LocalCloudConfigurator support system of the Eclipse Arrowhead 5.2.2 framework in `ah5-LocalCloudConfigurator-java` of the primary Java source tree (`Eclipse-Arrowhead-java`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 6
2.4 Deployment 6
2.5 Verification 6

3 Services **7**

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the LocalCloudConfigurator system implementation, which realizes the LocalCloud-Configurator SysD [1]. In Section 2, we describe implementation details of the system.

2 Implementation

The implementation is located in `ah5-LocalCloudConfigurator-java` (artifact `arrowhead-localcloudconfigurator`, `localcloudconfigurator.jar`). The same JAR runs as the Arrowhead system (`serve`, default), as the host worker (`worker`) and as the retained command-line tools (`keygen`, `register`, `executor`, `plan`).

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (Web, Security, Data JPA, Quartz), Log4j2 with the JDBC log table, MariaDB driver, BouncyCastle (ColonyOS signatures).
- Eclipse Arrowhead common library as a private vendored source copy (`vendor/`).
- Storage: MySQL 8.4 (private schema `ah_localcloudconfigurator`); the host worker uses files.
- Host worker: non-web Spring application with the JDK `HttpsServer`.

2.2 Functional properties implementation

Package `eu.arrowhead.localcloudconfigurator`:

Component	Responsibility
`LocalCloudConfigurator Main`	Command dispatch: Arrowhead system, `worker` (non-web application), tools.
`LocalCloudConfigurator SystemInfo`	System `LocalCloudConfigurator`; services `localCloud Configuration` and `generalManagement`.
`Catalog`	Loads and validates the catalog (names, ownership of databases, scripts, dependency cycles, defaults); dependency closure, protected components, catalog hash.
`Planner`	Plans: defaults and selection with dependencies in order, provision, start, policy, explicit stops in reverse order, optional verification; plan hash.
`Runner`	Runs one step after checking it against the catalog (owner, kind, script, catalog hash, protected components, running dependents, database order); `docker compose` argument lists; output bounded and redacted.
`DirectBackend`	Worker execution: one deployment at a time on a single thread, results written atomically after every step, state-directory lock, interrupted deployments marked failed at start-up and never replayed.
`CloudColony`, `colony/`	Legacy ColonyOS backend: signed RPC, chained `lcc.step` workflows, `lcc.status`.
`service/Deployment Coordinator`	Status, plan and serialized apply with the plan-hash check and the active-deployment guard; shared by the API (JPA store) and the worker (file store).
`service/Configuration Service`	Saved designs with validation and revision checks.
`service/WorkerClient`, `tls/MutualTls`	API side of the worker connection: mutual TLS with peer-name pinning and the CRL.

Component	Responsibility
`service/BackendConfig`, `ColonyBackend`	Catalog bean and backend selection (`lcc.backend`).
`jpa/`	Entities `SavedConfiguration` (`configuration`), `Deployment`, `ActiveDeployment`, `Logs`; `JpaDeploymentStore`.
`api/http/`	`LocalCloudConfigurationAPI` (strict JSON, 64 KiB, native status codes and error bodies), `GeneralManagementAPI`, response headers.
`security/`	`ConfiguratorAccessFilter` and `ConfiguratorAuthorizationService`: Sysop, whitelist or ConsumerAuthorization grant per operation.
`worker/`	`WorkerApplication` (beans of the worker) and `WorkerServer` (loopback HTTPS, client authentication required).

2.2.1 Database

Table	Columns
`configuration`	`id` (UUID), `name`, `systems`, `removals`, `layout` (JSON text), `reapply`, `smoke`, `revision`, `updated_at`.
`deployment`	`id`, `plan`, `request` (JSON text), `state`, `result` (last reported state and steps), `submitted_at`, `updated_at`.
`active_deployment`	At most one row (`id` 1): `deployment_id` of the deployment that blocks new ones until the backend reports `success` or `failed`.
`logs`	Log4j2 JDBC appender (`generalManagement get-log`).

`deployment/database/init.sql` creates the schema; `least-privilege.sh` restricts the application account to SELECT, INSERT, UPDATE and DELETE. Hibernate validates the schema at start-up.

2.2.2 Host worker

The worker is started as `java -jar localcloudconfigurator.jar worker --lcc...` by a per-user launchd service installed with `scripts/install-direct.py` (RunAtLoad, KeepAlive). It does not register in the ServiceRegistry; it is the configurator's local agent. Properties:

Property	Meaning
`lcc.worker.address`, `lcc.worker.port`	127.0.0.1, 8493; a non-loopback address is refused at start-up.
`lcc.worker.ssl.key-store`, `trust-store`, `*-password`	Worker certificate `LocalCloudConfiguratorWorker` (PKCS12), cloud truststore; passwords from a 0600 config tree.
`lcc.worker.ssl.crl-file`	Cloud CRL for client certificates (optional, recommended).
`lcc.worker.allowed-clients`	`LocalCloudConfigurator`.
`lcc.worker.cloud-dir`, `lcc.worker.docker`	Cloud view (`compose.yaml`, `scripts`, `.env`), Docker CLI.

Property	Meaning
`lcc.worker.state`, `lcc.catalog.file`	File state (`active-deployment.json`, `deployments/`, `workflows/`), the worker's own catalog copy.

Worker interface (HTTPS, client certificate required, JSON as in the IDD [2]):

Method	Path	Behaviour
GET	`/healthz`	`{"status": "ok", "executionBackend": "direct"}`.
GET	`/api/status`	Running Compose services and component states.
POST	`/api/plan`	Plan for a request against the worker's catalog.
POST	`/api/deployments`	Reviewed selection with `expectedPlanHash`; the worker rebuilds the plan and refuses a different hash (409); 202 with a `local-<uuid>` identifier.
GET	`/api/deployments/{id}`	State and step results of a worker deployment.

2.3 Non functional properties implementation

2.3.1 Security

Mutual TLS on 8492 and to the worker (SysD, Section Security); outsourced authentication and the access filter for every operation; security profile guard; DML-only database account; read-only root filesystem.

2.3.2 Power management

None. The `sysMonitor power` observation always reports line supply.

2.3.3 Internal monitoring

Logs to the console, a rolling file and the `logs` table; the worker logs to the console (launchd log files). sysMonitor observations every 30 s.

2.3.4 Configuration

Spring properties (SysD), secrets from `configtree:/run/secrets/`.

2.4 Deployment

`compose.yaml` (project `ah5-localcloudconfigurator`) runs `localcloudconfigurator` (image `arrowhead-localcloudconfigurator-standalone:5.2.2`, UID 10001, read-only root filesystem, port 8492 on the host's loopback by default, `host.docker.internal` mapped to the host) and `localcloudconfigurator-db` (MySQL 8.4.11 with TLS, internal network). `scripts/deploy.sh` puts both services into a cloud's Compose file and installs the worker; it changes a deployed cloud and is run only by an operator.

2.5 Verification

`mvn clean verify`: catalog, planner and runner tests; direct backend (persistence, no replay, single worker); deployment coordinator (guard across restarts, stale hash); a full-application MockMvc test of every `/api` route with identity and grant checks, size limit and strict JSON; a plan-to-deployment run against a worker over mutual TLS and refusal of a missing, foreign, wrongly named or revoked certificate; the security profile test. A gated test validates the schema against a fresh MySQL 8.4.11 container and checks the DML-only grants.

3 Services

Services produced	SysD ref	SD ref	IDD ref
`localCloudConfiguration`	[1]	[3]	[2]
`sysMonitor`	[1]	[4]	[5]
`generalManagement`	[1]	–	–
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
systemDiscovery, serviceDiscovery	[6]	[7]	–
identity	[8]	[9]	–
authorization (verify)	[10]	–	–

4 References

[1] J. Delsing, “LocalCloudConfigurator – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-LocalCloudConfigurator-java/documentation/v5.2.2/LocalCloudConfigurator-v5.2.2-SysD.pdf.

[2] ——, “localCloudConfiguration HTTP/TLS/JSON – Interface Design Description (LocalCloudConfigurator),” Eclipse Arrowhead, IDD 5.2.2, 2026, localCloudConfigurator-v5.2.2-localCloudConfiguration-HTTP-IDD.pdf.

[3] ——, “localCloudConfiguration – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudConfigurator-v5.2.2-localCloudConfiguration-SD.pdf.

[4] ——, “sysMonitor – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudConfigurator-v5.2.2-sysMonitor-SD.pdf.

[5] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (LocalCloudConfigurator),” Eclipse Arrowhead, IDD 5.2.2, 2026, localCloudConfigurator-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[6] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[7] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[8] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[9] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[10] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-09	5.2.2	Document created for the Java-Spring implementation: components, database, host worker (non-web Spring application, loopback 8493, mutual TLS, not registered), deployment and verification.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	